

Sommaire

Introduction	1
Origines et évolution des réseaux de neurones	1
1.1. Les débuts de l'intelligence artificielle	1
1.2. Le modèle de McCulloch et Pitts	1
1.3. Le perceptron de Rosenblatt	2
1.4. Le perceptron multicouche de Hinton	2
Fonctionnement des réseaux de neurones	2
2.1. Le neurone artificiel	2
2.2. Les poids, le biais et l'activation	2
2.3. La descente de gradient	4
2.4. La rétropropagation du gradient	4
Applications des réseaux de neurones	5
3.1. Le traitement des images	5
3.2. Exemple : reconnaissance botanique	5
3.3. Intérêt et limites de cette application	6
Limites des réseaux neuronaux	6
4.1. Besoin de grandes quantités de données	7
4.2. Coût computationnel	7
4.3. Problème de la boîte noire	7

4.4. Surapprentissage et difficultés d'optimisation	7
4.5. Solutions et pistes d'amélioration	7
Étude pratique : programme Python sur le jeu de données IRIS	7
5.1. Présentation du jeu de données	7
5.2. Construction du modèle avec TensorFlow/Keras	8
5.3. Entraînement et évaluation	8
5.4. Analyse des résultats : loss, accuracy, matrice de confusion, ROC	9-10
Conclusion	10
Sources	11

Rapport de TPE

Depuis 5 ans nous entendons parler des intelligences artificielles qui évoluent et deviennent de plus en plus réalistes et intelligentes représentant presque un danger pour certains emplois. Pourtant à notre niveau nous ne savons pas ce que toute la technologie qui se cache derrière le terme IA. Aujourd'hui dans ce rapport nous allons expliquer le fonctionnement des réseaux de neurones, indispensables à la compréhension des intelligences artificielles fonctionnant en machine learning.

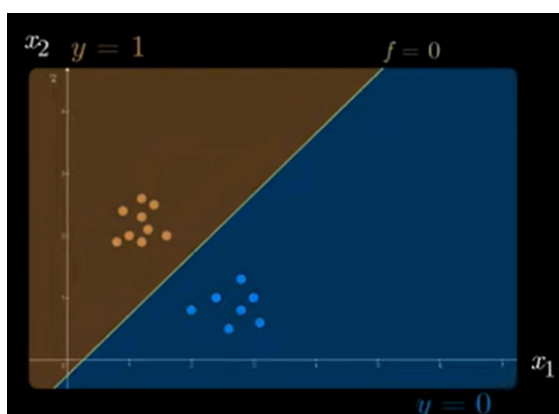
Il faut savoir que la première intelligence artificielle a été créée en 1956, cependant nous allons voir que les réseaux neuronaux ont été découverts avant cela mais ce n'est qu'avec le développement rapide de la puissance de calcul des ordinateurs et la découverte d'outils mathématiques utiles à ces IA que ceux-ci se sont développés.

Nous allons donc expliquer dans ce rapport dans quelle mesure la compréhension du fonctionnement des réseaux neuronaux permet-elle d'expliquer leur efficacité et leurs nombreuses applications ?

Nous allons donc dans un premier temps présenter les réseaux de neurones, ensuite nous expliquerons leur fonctionnement avant de donner des exemples d'applications et les limites auxquelles les réseaux neuronaux font face. Enfin nous expliquerons rapidement le code créé pour ajouter des précisions.

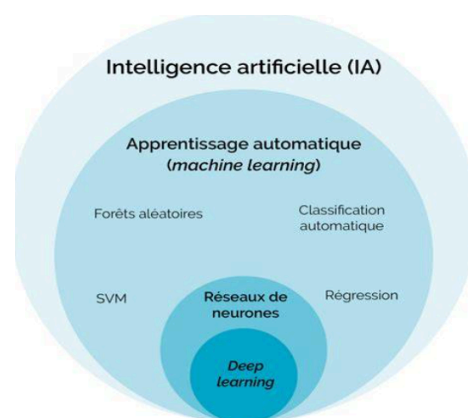
Avant d'expliquer comment fonctionnent les réseaux de neurones nous allons dans un premier temps présenter d'où ils viennent, quand ils sont apparus et pourquoi.

Depuis les années 30 les scientifiques cherchent à développer un type de machine capable de raisonner afin de résoudre des problèmes plus complexes ce qui a conduit à la naissance des intelligences artificielles. Il faut savoir que le domaine de l'intelligence artificielle est très vaste et comporte de nombreux sous-domaines comme le machine learning. Le machine learning est une technique qui permet à



un ordinateur de s'améliorer au fur et à mesure du temps. Dans cette spécialisation se trouvent de nombreuses applications.

En effet pour réaliser des tâches complexes les machines doivent développer un modèle qui corresponde le plus possible aux données transmises à la machine. Pour le rendre plus précis et fiable, les scientifiques conçoivent un algorithme d'optimisation afin de minimiser la distance entre le modèle et les données.



De nombreux algorithmes d'optimisation existent comme les modèles linéaires ou encore les arbres de décision qui viennent chacun avec leur algorithme d'optimisation.

Un des domaines particuliers du machine learning est le deep learning. Pour expliquer ce que c'est nous devons faire un petit peu d'histoire et retourner dans les années 40. En 1943 exactement les scientifiques McCulloch et Pitts créent un réseau de neurones artificiels en imitant les neurones humains. En effet, au lieu

d'utiliser une unique fonction afin de s'adapter aux données, les réseaux de neurones utilisent une fonction par neurone.

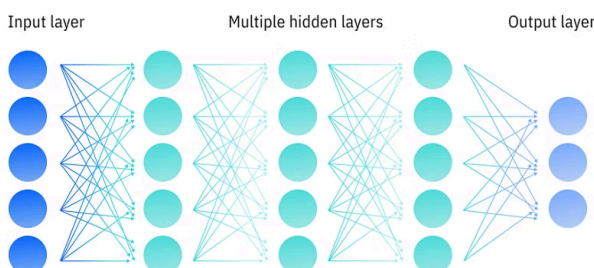
À l'aide de ces neurones artificiels McCulloch et Pitts réussirent à recréer les portes logiques AND et OR directement avec les réseaux neuronaux, chose qui n'avait encore jamais été possible de cette manière. Dans le même temps, ces deux scientifiques annoncent qu'en branchant plusieurs neurones à la manière du corps humain nous serions capables de résoudre n'importe quel problème de logique booléenne c'est-à-dire tous les problèmes qui n'ont que deux valeurs 0 ou 1. Comme vous le pensez après cette annonce, il y a eu un engouement démesuré qui peut être expliqué par la nouveauté de la technologie et les promesses faites par McCulloch et Pitts.

Les gens pensaient que dans les années suivantes ils pourraient développer des intelligences artificielles capables de remplacer totalement les êtres humains. Cependant ce système a des failles, par exemple le fait qu'il n'y ait pas de fonction d'apprentissage, ainsi les scientifiques devaient trouver eux-mêmes les coefficients de la fonction de chaque neurone.

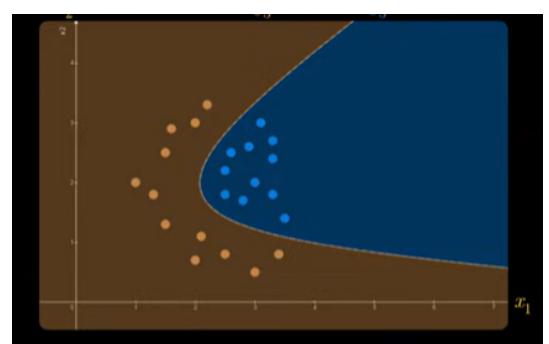
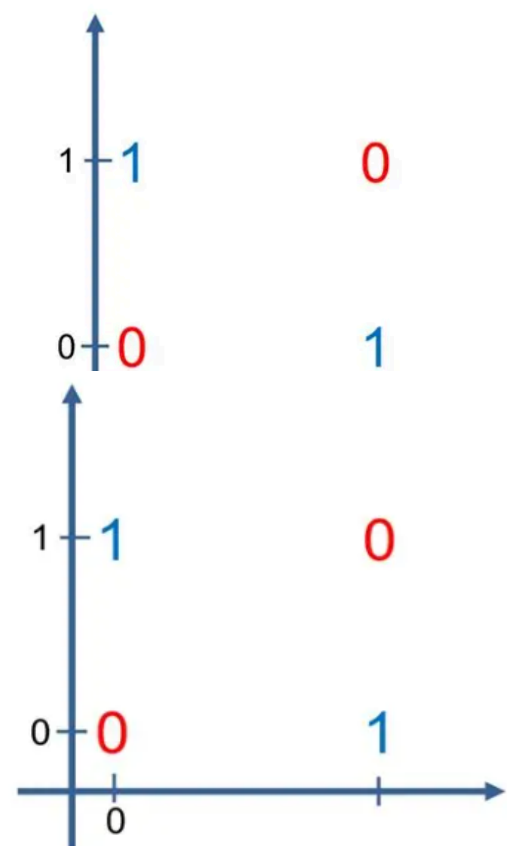
Les années passent et en 1957 un psychologue américain nommé Frank Rosenblatt trouva comment améliorer le modèle de McCulloch et Pitts en proposant le premier algorithme d'apprentissage de l'histoire du deep learning lui permettant de trouver les valeurs des poids de la fonction afin d'obtenir les sorties qui conviennent. Pour réaliser ceci s'est inspiré de la théorie de Hebb qui dit que lorsque deux neurones sont excités conjointement cela renforce leur lien synaptique c'est-à-dire qu'ils renforcent les connexions qui les unissent au fur et à mesure du temps. Ainsi Frank Rosenblatt a trouvé l'idée d'entraîner un neurone artificiel sur des données de référence pour que celui-ci renforce ses paramètres à chaque fois qu'une entrée est activée en même temps que la sortie présente dans les données de référence. Mais encore une fois ce modèle a été surestimé. En effet à cette époque on pensait qu'avec le perceptron les humains seraient capables de construire des machines capables de lire, parler, marcher et même d'avoir une conscience, c'est dire l'engouement qu'il y a eu pour ce modèle.

Cependant, après quelques années, il est apparu que ces promesses ne pourraient pas être tenues, car le perceptron est un modèle linéaire. C'est une grande limite car de nombreux problèmes ne sont pas linéaires et échappent donc à cette approche. C'est le cas du XOR, en effet comme nous pouvons le

voir sur ce graphique il n'est pas possible de séparer les 1 des 0 à l'aide d'une simple droite. Cette désillusion a conduit au premier hiver de



l'intelligence artificielle, entre 1974 et 1980, période durant laquelle les investisseurs ont perdu confiance dans ce domaine.



D'autres problèmes ont mené à une stagnation de la technologie IA par exemple le manque de puissance de calcul. En effet à cette époque les ordinateurs n'étaient pas encore très puissants s'agissant d'une nouvelle science de plus les calculs nécessaires aux réseaux de neurones étaient trop coûteux. Dernier problème contrairement à aujourd'hui il n'y avait pas de bases de données donc les modèles ne pouvaient pas apprendre efficacement.

Ce n'est qu'en 1986 que Geoffrey Hinton développe le perceptron multicouche, le premier réseau de neurones artificiels comme on l'entend. En effet, Geoffrey Hinton reprend l'idée de McCulloch et Pitts de connecter plusieurs neurones entre eux, afin d'obtenir un modèle non linéaire. Par exemple en utilisant 3 neurones répartis en deux couches : une couche d'entrée et une couche de sortie. Le plus intéressant avec ce modèle est qu'au lieu d'utiliser 3 neurones nous pouvons en ajouter autant que nous voulons que ce soit en neurones d'entrées ou ce que l'on appelle les neurones cachés neurones permettant de modifier la droite pour la courber et l'adapter selon le type de problème. Les neurones dits "cachés" effectuent leurs opérations sur les données transmises par les neurones précédents. Le point fort de cette découverte est que nous pouvons rajouter à nos modèles autant de couches cachées que nous le voulons.

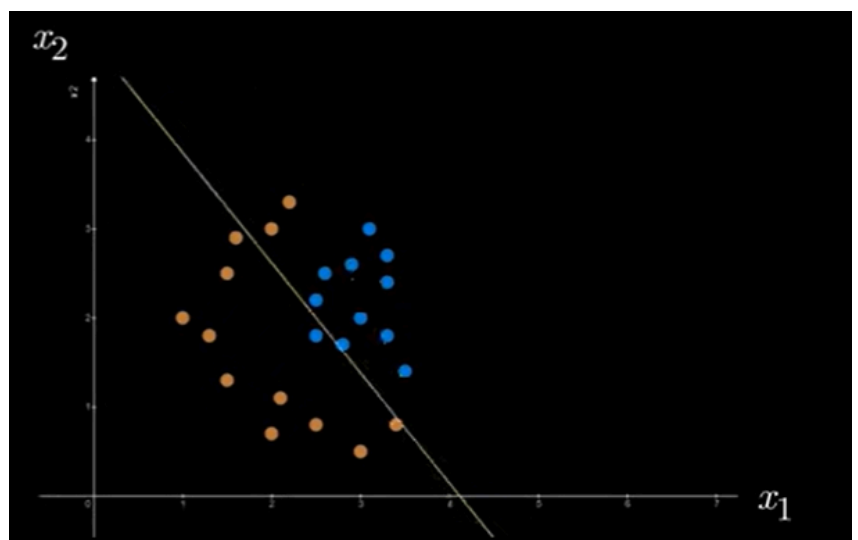
Plus il y a de neurones et de couches cachées plus le modèle est complexe et intéressant.

Cependant il reste un problème, comment faire pour entraîner ce réseau de neurones automatiquement car il n'est pas tenable de trouver à la main la valeur de tous les paramètres de chaque neurone. Geoffrey Hinton utilise alors une technique nommée backpropagation afin de calculer l'erreur et de pouvoir la diminuer. Nous la détaillerons en profondeur plus loin. Le modèle de Hinton reste alors le summum de la technologie pendant de nombreuses années. Le domaine de l'intelligence artificielle retombant dans un hiver qui peut cette fois être expliqué par les limites des ordinateurs et des données présentes à cette époque.

Ce n'est que dans les années 90 que l'on a commencé à développer les premières variantes du perceptron multicouche avec par exemple Yann LeCun qui inventa les premiers réseaux de neurones convolutifs capables de reconnaître et traiter des images puis plus tard la reconnaissance vocale. Au fur et à mesure de l'avancement d'internet les intelligences artificielles se développent et l'explosion due au réseau sociaux a fait que cette technologie a explosé ces 10 dernières années. Nous avons donc vu dans cette première partie l'histoire des réseaux de neurones mais sans en détailler le fonctionnement.

Dans cette seconde partie nous allons donc étudier le fonctionnement des réseaux de neurones. Nous allons partir du modèle le plus simple mais comme nous l'avons vu précédemment les progrès de l'intelligence artificielle ne reposent pas à des refontes totales de la technologie mais plutôt à un ajout continu d'amélioration menant à la technologie que nous connaissons.

Retournons donc au début de la technologie avec les neurones de McCulloch et Pitts. Comme expliqué précédemment, ils ont imité les neurones humains en transposant ce comportement à une machine en utilisant une fonction. Celle-ci prend en entrée des signaux X et retourne une sortie Y . Cependant ce n'est pas qu'une simple fonction, celle-ci contient deux étapes indispensables. Tout d'abord on fait la somme de toutes les entrées du neurone en les multipliant par un coefficient w qui est appelé le poids et représente l'activité synaptique du neurone. Le poids de chaque neurone peut être différent, il peut même être négatif si le neurone est inhibiteur. Ainsi nous obtenons la fonction f déterminée par $f = w_1.x_1 + w_2.x_2 + w_3.x_3$ avec autant de termes que d'entrées rattachées au neurone



En réalité, cette opération correspond à un produit scalaire entre un vecteur d'entrée, qui représente les données, et un vecteur de poids, qui représente les paramètres du modèle. Cette écriture permet de donner une interprétation géométrique au neurone : celui-ci mesure en quelque sorte l'alignement entre les données d'entrée et les caractéristiques qu'il a apprises. Plus cet alignement est important, plus la sortie du neurone sera élevée. On peut ainsi se dire qu'un neurone se comporte comme un détecteur de motifs, capable de reconnaître certaines caractéristiques dans les données, comme par exemple des formes ou des textures dans une image.

Une fois cette étape réalisée la fonction effectue la phase d'activation, c'est-à-dire qu'on regarde le résultat du calcul précédent et si le résultat dépasse un seuil qui est souvent 0 alors le neurone s'active et renvoie la valeur 1 sinon il reste à 0.

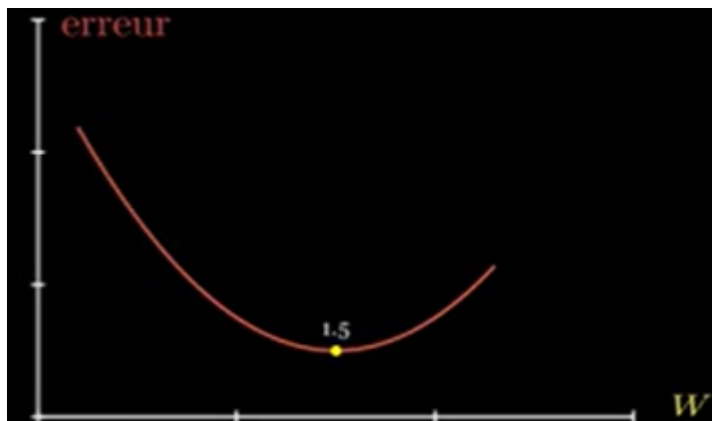
Au lieu de faire ça nous pouvons également appliquer un biais à la fonction afin d'indiquer à quel point la somme doit être élevée pour que le neurone soit activé.

Il s'agit là de la base du deep learning.

Passons maintenant au perceptron de Frank Rosenblatt qui réutilise les neurones artificiels de McCulloch et Pitts mais en ajoutant un algorithme d'apprentissage automatique à l'aide de données de référence. Frank Rosenblatt crée alors une formule permettant au neurone de renforcer ses paramètres w à chaque fois qu'une entrée X est activée en même temps que la sortie Y se situant dans les données. Les paramètres W sont alors mis à jour en calculant la différence entre la sortie de référence et la sortie produite par le neurone et en multipliant cette différence par la valeur de chaque entrée X , on obtient donc $W = W + \eta(y - y')X$ avec y la sortie de référence, y' la sortie renvoyée par le neurone et η la vitesse d'apprentissage. La vitesse d'apprentissage représente un point clé du perceptron. En effet si elle est choisie trop faible le neurone apprend très lentement et si on la choisit trop élevée alors le neurone peut sauter autour de la solution sans jamais l'atteindre. En fait ce qu'il se passe dans la fonction c'est que si jamais la valeur obtenue n'est pas la même que la valeur de référence alors le poids se verra être additionner avec la vitesse d'apprentissage (aussi appelée le pas) ainsi cela augmentera la fonction de référence la combinaison linéaire des entrées du neurone lui permettant de se rapprocher de sa valeur d'activation. Cette fonction implique que si modèle se trompe, c'est-à-dire que la valeur renvoyée par la fonction n'est pas celle attendue, alors l'algorithme se

corrige au fur et à mesure du temps. Plus l'erreur est importante plus la correction est grande.

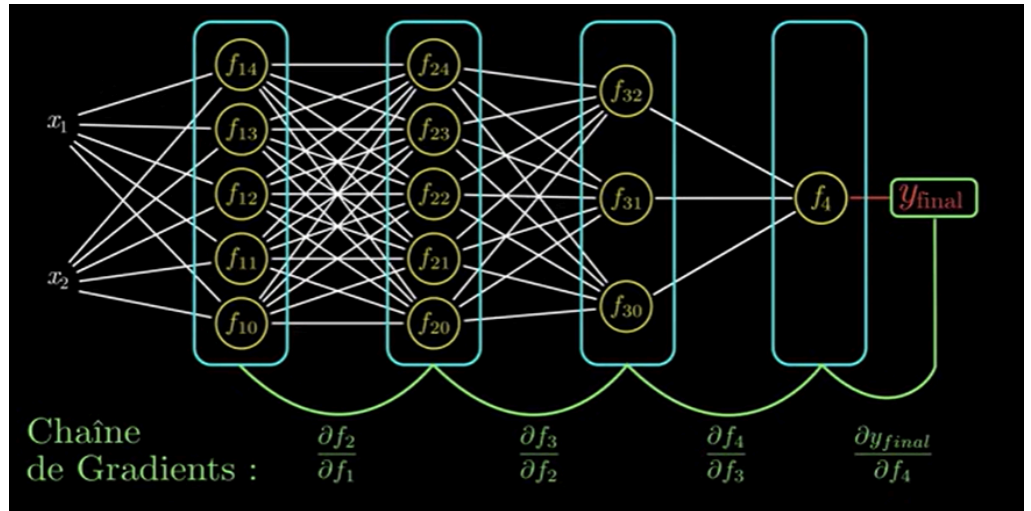
À chaque itération, les paramètres sont ajustés dans la direction qui permet de réduire l'erreur du modèle. Autrement dit, l'algorithme "se déplace" dans cet espace afin de trouver une configuration des poids qui minimise la fonction d'erreur. Ce processus itératif permet ainsi d'améliorer progressivement les performances du réseau. Le but de cette fonction est de trouver le minimum d'une fonction (la fonction d'erreur) et de modifier la valeur du poids en conséquence. La formule de la descente de gradient est donc donnée par $w = w - \eta \nabla E$ avec w le



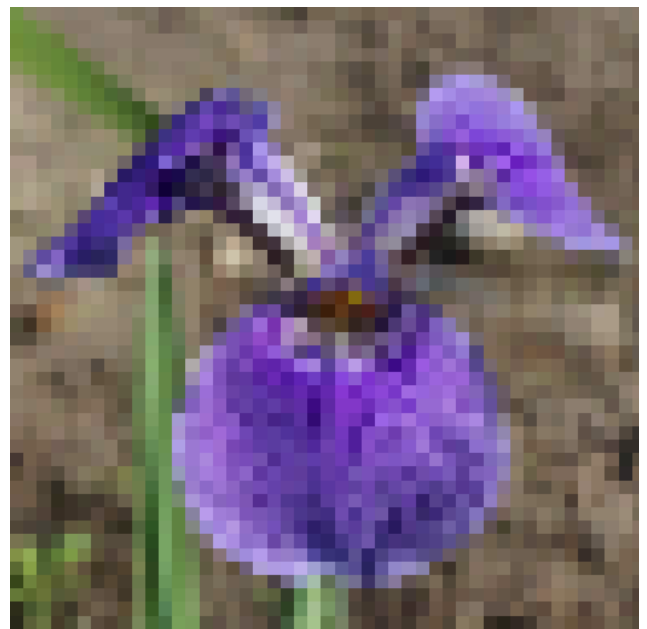
poids du neurone, E l'erreur calculée avec la valeur attendue soustraite à la valeur obtenue et la vitesse d'apprentissage. La dérivée partielle joue un rôle essentiel dans ce processus, car elle permet de mesurer la variation de l'erreur en fonction de chaque paramètre. Elle indique dans quelle direction il faut modifier les poids pour réduire l'erreur, ainsi que l'intensité de cette modification. Cette information est utilisée dans la descente de gradient pour ajuster efficacement les paramètres du modèle. Si le résultat du gradient est négatif, l'erreur diminue ce qui est positif.

Le principe de la descente de gradient est de répéter le processus un grand nombre de fois car il compense

ses erreurs au fur et à mesure. Les réseaux neuronaux fonctionnent alors par itérations : à chaque itération d'entraînement on se sert des données de référence pour trouver le résultat attendu. Le neurone renvoie alors une valeur en fonction des données entrées, on calcule l'erreur puis le gradient qui nous dit si nous sommes loin de la valeur de référence. On modifie alors les poids du neurone suivant la descente de gradient puis on recommence.



Geoffrey Hinton en plus d'adapter la descente de gradient à l'utilisation des réseaux neuronaux trouve également une méthode fondamentale et extrêmement efficace pour entraîner les réseaux de neurones. La backpropagation (ou rétropropagation du gradient). Son objectif est de permettre l'ajustement automatique de tous les poids du réseau afin de réduire l'erreur entre la prédiction du modèle et la valeur réelle attendue. Le processus commence par une application de la fonction du neurone: les données entrent dans le réseau par la couche d'entrée, puis sont transformées successivement par chaque couche grâce aux poids et aux fonctions d'activation, jusqu'à produire une sortie finale. Cette sortie est ensuite comparée à la valeur attendue à l'aide d'une fonction de coût, qui mesure l'écart et quantifie l'erreur globale du modèle. Une fois cette erreur calculée, la backpropagation intervient dans une deuxième phase : l'erreur est alors propagée en sens inverse, de la couche de sortie vers les couches précédentes, afin de déterminer la contribution de chaque neurone à l'erreur finale. C'est-à-dire que nous remontons le réseau de neurone avec la valeur de l'erreur et nous effectuons les opérations à rebours pour déterminer à quel point tel neurone a contribué à l'augmentation de l'erreur. L'idée centrale est que chaque couche reçoit une part de l'erreur en fonction de son influence sur le résultat, ce qui permet de savoir quels paramètres sont responsables des mauvaises prédictions. Cette opération montre alors dans quelle direction et avec quelle intensité il faut modifier les paramètres pour réduire l'erreur. Ensuite, ces gradients sont utilisés avec la descente de gradient pour mettre à jour progressivement les poids du réseau : les paramètres ayant fortement contribué à l'erreur sont davantage corrigés, tandis que ceux ayant peu d'impact le sont légèrement. L'ensemble du processus est répété sur de nombreuses données d'entraînement, ce qui permet au réseau d'améliorer progressivement ses performances. La backpropagation est ainsi essentielle car elle rend possible l'apprentissage des réseaux multicouches en automatisant la répartition de l'erreur et le calcul des corrections, ce qui serait impossible à faire manuellement. Nous avons donc vu dans cette partie qu'un réseau de neurones peut être vu comme une fonction mathématique paramétrée qui transforme des données d'entrée en une prédiction, et dont les paramètres sont ajustés afin de minimiser une fonction



d'erreur cependant nous n'avons pas encore traité à quoi servent les réseaux neuronaux et quelles applications pouvons nous en tirer.

Dans cette troisième partie nous allons donc nous pencher sur l'étude d'exemples concrets des réseaux neuronaux car ceux-ci ont un réel intérêt comme nous allons le constater. Vous pensez bien que les réseaux de neurones ne sont pas uniquement des modèles théoriques : ils sont aujourd'hui utilisés dans de nombreux domaines concrets, notamment dans la reconnaissance d'image. Un exemple particulièrement intéressant est leur utilisation en botanique, où ils permettent d'identifier automatiquement des espèces de plantes à partir de photographies. Cette application illustre parfaitement l'efficacité des réseaux de neurones ainsi que leur fonctionnement dans une situation réelle. Avant de réfléchir à ces exemples, il faut savoir comment une image est traitée par un réseau de neurones. Contrairement à ce que l'on pourrait penser, une image n'est pas directement "vue" comme on l'entend. Elle est transformée en une espèce de grille de pixels, où chaque pixel est associé à une valeur numérique représentant une couleur ou une intensité lumineuse. Ainsi, une image devient une grande matrice de nombres que le réseau va pouvoir exploiter. Par exemple, une image de feuille peut être représentée par des milliers de valeurs correspondant à ses couleurs et ses formes. Pour analyser ces images, on utilise généralement un type particulier de réseau de neurones appelé réseau de neurones convolutif. Ce type de réseau est spécialement conçu pour traiter des données visuelles. Son fonctionnement repose sur plusieurs couches successives qui vont extraire progressivement des informations de plus en plus complexes à partir de l'image. Dans les premières couches, le réseau détecte des éléments simples comme des contours, des lignes ou des variations de couleur. Dans les couches suivantes, il combine ces éléments pour reconnaître des formes plus complexes, comme des textures ou des motifs spécifiques. Enfin, dans les dernières couches, le réseau est capable d'identifier des structures complètes comme une feuille, une fleur ou un type de plante. Dans le cas de la botanique, le réseau de neurones est entraîné à partir d'un grand nombre d'images de plantes, chacune étant associée à une étiquette correspondant à son espèce. Par exemple, une base de données peut contenir des milliers de photos de feuilles de chêne. Pendant l'entraînement, le réseau reçoit une image, produit une prédiction, puis compare cette prédiction avec la bonne réponse. Grâce à la backpropagation et à la descente de gradient, il ajuste progressivement ses paramètres afin de réduire ses erreurs. Ce processus est alors répété un grand nombre de fois, ce qui permet au réseau d'apprendre à reconnaître les caractéristiques propres à chaque espèce. Une fois entraîné, le réseau peut être utilisé dans des situations réelles. Par exemple, un botaniste ou un amateur peut prendre une photo d'une plante avec son téléphone et la soumettre à l'intelligence artificielle. Celle-ci analyse alors l'image en quelques secondes et propose une ou plusieurs espèces probables. Cette technologie est déjà utilisée dans certaines applications mobiles qui permettent d'identifier des plantes en temps réel. Les intelligences artificielles permettent alors un gain de temps considérable, car l'identification d'une plante peut être longue et nécessite souvent une expertise approfondie. Ensuite, ils permettent de réduire les erreurs humaines, notamment dans le cas d'espèces très similaires visuellement. Enfin, ils rendent la botanique plus accessible en permettant à des non-spécialistes d'obtenir rapidement des informations fiables. Cependant, cette application montre également certaines limites des réseaux de neurones. Par exemple, la qualité des résultats dépend fortement des données d'entraînement : si certaines espèces sont sous-représentées dans la base de données, le modèle aura du mal à les reconnaître. De plus, des conditions particulières comme une mauvaise luminosité, un angle de prise de vue inhabituel ou une plante partiellement cachée peuvent perturber le réseau. Cela montre que, même s'ils sont performants, ces systèmes ne sont pas infallibles et



doivent être utilisés avec précaution. Cet exemple met donc en évidence le lien direct entre le fonctionnement théorique des réseaux de neurones et leurs applications concrètes. Les notions étudiées précédemment, comme les neurones artificiels, la descente de gradient ou la backpropagation, prennent ici tout leur sens : elles permettent au réseau d'apprendre à partir de données et de s'améliorer progressivement. Ainsi, la reconnaissance d'image en botanique constitue un exemple clair et concret de l'efficacité des réseaux de neurones, tout en illustrant leurs limites et les enjeux liés à leur utilisation.

Nous allons donc étudier dans cette quatrième parties les limites auxquelles les réseaux neuronaux et plus largement les intelligences artificielles font face. Les réseaux de neurones bien que très précis après de nombreuses itérations visant à l'apprentissage ne seront jamais qu'une approximation. En effet le réseau neuronal ne transmet pas la vérité de la fonction mais plutôt un modèle visant à approximer ainsi tous les réseaux neuronaux font des erreurs et même si le taux d'erreur est de 0.04% ce n'est pas une parfaite précision. Les réseaux de neurones, bien qu'ils soient aujourd'hui au cœur de nombreuses avancées en intelligence artificielle, présentent plusieurs limites importantes qui expliquent à la fois leurs contraintes d'utilisation et la nécessité de techniques d'amélioration. Tout d'abord, leur fonctionnement repose sur l'utilisation d'un très grand nombre de paramètres, ce qui implique qu'ils ont besoin d'une quantité massive de données d'entraînement pour apprendre correctement. Sans suffisamment d'exemples, le modèle ne parvient pas à généraliser et produit des résultats peu fiables, ce qui limite leur efficacité dans des contextes où les données sont rares ou difficiles à obtenir. Ensuite, l'entraînement de ces modèles est particulièrement coûteux en ressources de calcul, car il nécessite d'effectuer un très grand nombre d'opérations mathématiques (notamment des multiplications de matrices) sur de grandes quantités de données, ce qui rend leur apprentissage long et énergivore. C'est pour cette raison que l'on utilise des processeurs spécialisés comme les GPU afin d'accélérer ces calculs. Une autre limite majeure est le problème de la boîte noire : même si un réseau de neurones peut atteindre d'excellentes performances, il est souvent très difficile d'expliquer précisément comment il a pris une décision, car celle-ci résulte de millions d'opérations réparties dans plusieurs couches. Cela pose un problème important dans des domaines sensibles comme la médecine, la finance ou la justice, où l'applicabilité des décisions est essentielle. Par ailleurs, les réseaux de neurones peuvent souffrir du phénomène de sur apprentissage (overfitting), qui se produit lorsque le modèle apprend trop précisément les données d'entraînement, au point de les mémoriser plutôt que d'en extraire des tendances générales. Dans ce cas, il obtient de très bons résultats sur les données connues mais échoue sur de nouvelles données, ce qui réduit fortement sa capacité de généralisation. Il existe également des problèmes techniques liés à l'optimisation, notamment la disparition ou l'explosion du gradient, qui peuvent apparaître dans les réseaux très profonds et rendre l'apprentissage instable ou inefficace. Face à ces différentes limites, plusieurs solutions ont été développées. Pour limiter le sur apprentissage, on peut augmenter la quantité de données disponibles, mais aussi utiliser des techniques comme la régularisation, qui



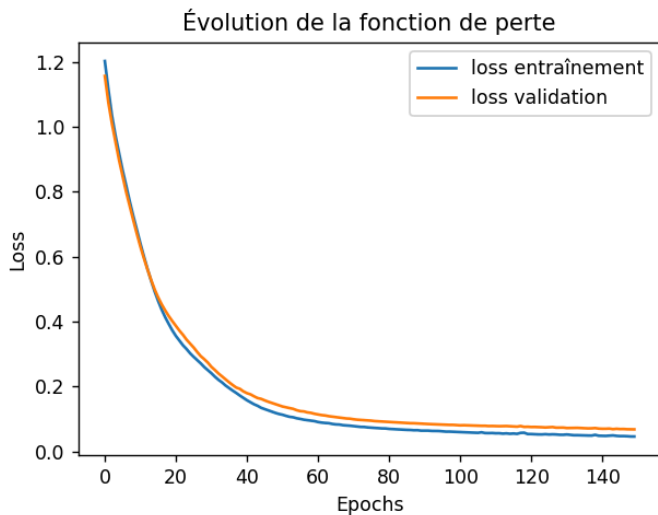
ajoute une contrainte pour éviter que les poids deviennent trop grands, ou le dropout, qui consiste à désactiver aléatoirement certains neurones pendant l'entraînement afin de rendre le réseau plus robuste et moins dépendant de chemins spécifiques. Pour améliorer la stabilité et la vitesse d'apprentissage, on utilise des algorithmes d'optimisation plus avancés que la descente de gradient classique, comme Adam ou RMS Prop, qui adaptent automatiquement la vitesse d'apprentissage en fonction des paramètres. En parallèle, les progrès matériels jouent aussi un rôle essentiel, avec l'utilisation de cartes graphiques (GPU) et de processeurs spécialisés (TPU) qui permettent d'accélérer considérablement les calculs nécessaires à l'entraînement. Enfin, pour répondre au problème de la boîte noire, un domaine de recherche appelé IA explicable (XAI) tente de rendre les décisions des modèles plus compréhensibles en identifiant les variables les plus influentes ou en simplifiant certaines parties du réseau. Ainsi, même si les réseaux de neurones présentent des limites importantes liées aux données, au calcul, à l'interprétabilité et à la stabilité de l'apprentissage, de nombreuses solutions techniques et scientifiques ont été développées pour améliorer leur efficacité, leur robustesse et leur utilisation dans des domaines de plus en plus variés.

```
# -----  
# 4. Découpage entraînement / test  
# -----  
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled,  
    y_cat,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)
```

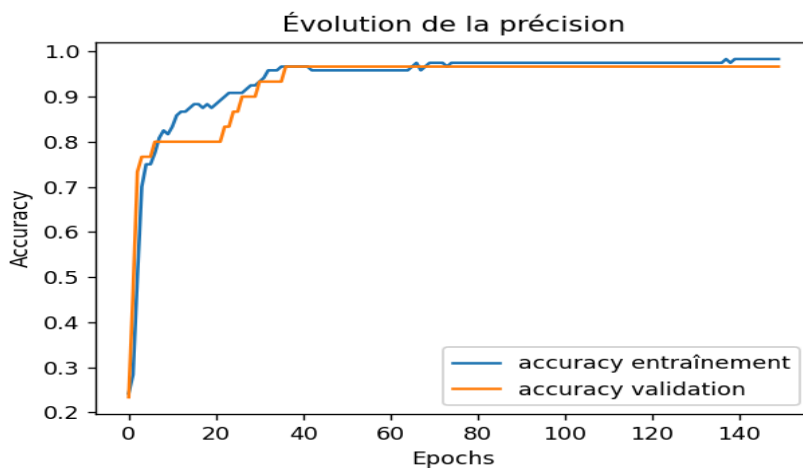
Afin d'illustrer concrètement le fonctionnement d'un réseau de neurones, nous avons réalisé un programme en Python à l'aide de TensorFlow et Keras sur le jeu de données IRIS. Ce jeu de données contient 150 fleurs décrites par 4 caractéristiques, que le réseau utilise comme données d'entrée pour apprendre à reconnaître trois espèces différentes : setosa, versicolor et virginica. Dans un premier temps, les données sont chargées puis normalisées afin que toutes les valeurs aient la même échelle, ce qui facilite l'apprentissage du modèle. Les étiquettes sont ensuite transformées sous une forme adaptée à Keras, puis l'ensemble est séparé en données d'entraînement et en données de test pour vérifier si le modèle sait bien généraliser à partir d'exemples qu'il n'a pas vus. Le réseau est ensuite construit sous forme de modèle séquentiel, avec une couche d'entrée, deux couches cachées utilisant la fonction d'activation ReLU, puis une couche de sortie de trois neurones avec softmax, qui permet d'obtenir une probabilité pour chaque classe. Avant l'entraînement, le modèle est compilé avec l'optimiseur Adam, la fonction de perte

```
# -----  
# 5. Construction du réseau de neurones  
# Réseau dense à 2 couches cachées, comme décrit dans le RI  
# -----  
model = Sequential([  
    Input(shape=(4,)),  
    Dense(10, activation='relu'),  
    Dense(8, activation='relu'),  
    Dense(3, activation='softmax')  
)
```

categorical_crossentropy et la métrique accuracy, afin de mesurer ses performances et de corriger progressivement ses erreurs. L'apprentissage est ensuite réalisé pendant 150 epochs : à chaque passage, le réseau compare ses prédictions aux vraies réponses, calcule l'erreur puis ajuste ses poids grâce à la rétropropagation. Afin de vérifier les performances du modèle, nous avons utilisé 80% de l'échantillon pour l'entraînement et conservé les 20% restants pour le test, ce qui permet de comparer les résultats du réseau sur des données nouvelles. Une fois l'entraînement terminé, le programme évalue le modèle sur l'ensemble de test et affiche plusieurs outils d'analyse permettant de comprendre son comportement.

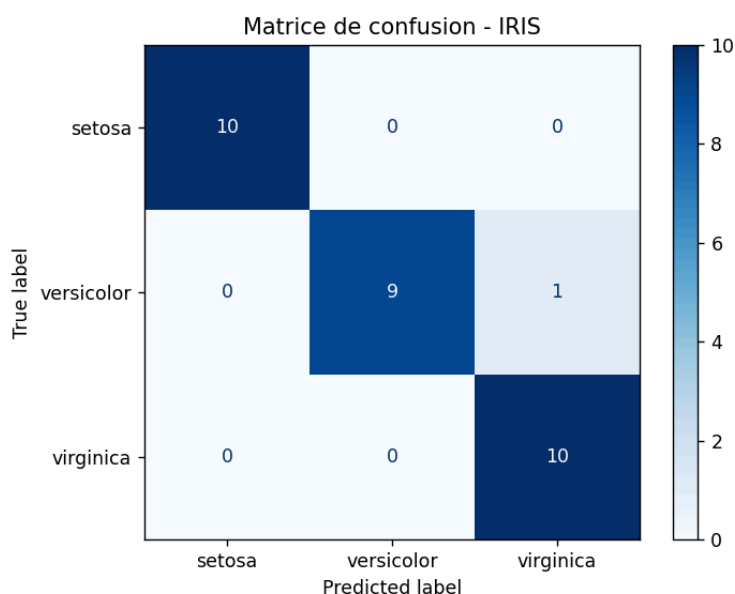


On peut voir sur la courbe de loss que l'erreur diminue au fil des epochs, ce qui montre que le réseau apprend de mieux en mieux. Au départ la loss est assez élevée car le modèle fait encore beaucoup d'erreurs et n'a pas encore trouvé les bons réglages pour ses paramètres. En quelques itérations cette valeur baisse nettement, montrant que les poids du réseau sont ajustés dans le bon sens par la rétropropagation. On remarque également que la loss sur l'ensemble de validation suit globalement la même tendance que celle d'apprentissage. C'est un bon signe puisqu'il indique que le modèle n'apprend pas les données d'entraînement par cœur mais a également la capacité d'aujourd'hui généraliser.

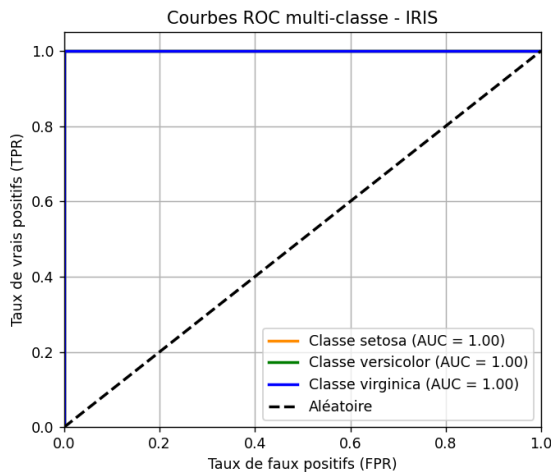


On peut en même temps observer la courbe d'accuracy qui nous montre que la précision des prédictions augmente, ce qui est cohérent avec la diminution de loss. Au début, le réseau reconnaît encore peu les classes, mais sa précision monte très rapidement lors des premières epochs puis se stabilise à un niveau élevé. Cela signifie que le modèle est de plus en plus capable de relier correctement les caractéristiques d'une fleur avec la bonne espèce. Le fait que l'accuracy de validation soit proche de celle d'apprentissage montre aussi

que le réseau ne surentraîne pas, ce qui est une bonne chose et va dans le sens de la fiabilité du modèle.



La matrice de confusion nous montre quelles classes sont bien reconnues mais aussi quelles classes sont parfois confondues entre elles. On peut voir que des espèces comme setosa sont particulièrement bien identifiées puisqu'elles se distinguent assez facilement des autres échantillons du jeu de données IRIS. A l'inverse, des confusions existent entre versicolor et virginica ce qui est en partie lié au fait que ces deux classes ont des caractéristiques plus proches. De cette façon, une matrice permet de voir si un modèle est bon, mais également où il se trompe : c'est plus détaillé qu'un simple taux de succès.



La courbe ROC indique la performance d'un réseau de neurones à extraire une classe de données parmi l'ensemble des classes. Une courbe proche du coin supérieur gauche correspond à un très bon classifieur. Plus une courbe est éloignée de la ligne représentative d'un tirage aléatoire, plus le modèle est apte à séparer une classe du reste. Ici, les courbes ROC sont très proches du coin supérieur gauche, ce qui signifie que le réseau de neurones parvient à bien distinguer les différentes espèces. Les scores d'AUC sont eux aussi très élevés, et tendent à dire que le réseau est un excellent classifieur.

À la fin de l'apprentissage, le modèle fait une loss sur l'ensemble de test de 0.0677, ce qui signifie que l'erreur est faible, et une précision de 96.67%, ce qui signifie que le modèle classe correctement la grande majorité des fleurs du test. Après tout, ce

programme illustre bien l'apprentissage d'un réseau de neurones artificiel : apprendre à partir d'exemples, améliorer ses résultats en corrigeant ses erreurs, puis être capable de classer de nouvelles données avec une bonne précision.

```
Loss sur le test : 0.0677
Accuracy sur le test : 96.67%
1/1 ————— 0s 54ms/step
```

Pour conclure, on peut dire qu'un modèle de réseau de neurones en intelligence artificielle est un ensemble de neurones artificiels organisés en couches, qui a la capacité de transformer des données d'entrée en une prédiction par le biais de calculs successifs faisant intervenir des poids, des biais et des fonctions d'activation. On parle de réseaux de neurones en machine learning car ces derniers apprennent depuis des exemples : le modèle compare les résultats qu'il prédit à ceux qui sont attendus, calcule un certain type d'erreur, puis fait un ajustement des paramètres du modèle à l'aide de la descente de gradient et de la rétropropagation afin de réduire l'erreur progressivement et de s'améliorer.

La présente étude s'est donc attachée à démontrer que la force de réseaux de neurones était de pouvoir apprendre automatiquement des relations complexes dans les données, là où des modèles plus simples parviennent rapidement à des limites. Cette force explique leur utilisation dans de nombreux domaines, comme la reconnaissance d'images, mais aussi leur importance grandissante dans l'intelligence artificielle telle que développée actuellement.

Mais ce fonctionnement a également ses limites : les réseaux de neurones ont souvent besoin de beaucoup de données d'entraînement, de grandes puissances de calcul et restent parfois très opaques. Ainsi, comprendre leur fonctionnement permet non seulement de mieux comprendre pourquoi ils sont aussi puissants, mais aussi de prendre du recul sur leurs limites et les précautions qu'il faut prendre lorsqu'on les utilise.

Sources:

- [A. Jung, "Machine Learning: The Basics," Springer, Singapore, 2022](#)
- [What is Machine Learning ? - IBM Technology](#)
- [Complete Machine Learning in One Video | Machine Learning Tutorial For Beginners 2026 | Simplilearn](#)
- [What is Machine Learning \(ML\) ?, Dave Bergmann](#)
- [Step by step guide to becoming a Machine Learning Engineer in 2026](#)
- [Jodouin, J.-F. \(1994\). *Les réseaux de neurones : principes et définitions*. Hermès.](#)
- [Faure, A. \(2006\). *Classification et commande par réseaux de neurones*. Hermes science publ. Lavoisier.](#)
- [Géron, A., & Bohy, A. \(2017\). *Machine learning avec Scikit-Learn*. Dunod.](#)
- [Ameisen, E., & Rougé, D. \(2020\). *Développer des applications machine learning*. First interactive.](#)